INTRODUCTION TO SOFTWARE ARCHITECTURE

SE3100 - Architecture Based Development

Vishan Jayasinghearachchi

Lecturer - Department of Software Engineering,

Faculty of Computing,

Sri Lanka Institute of Information Technology.

vishan.j@sliit.lk

LEARNING OUTCOMES

After completing this lecture, you will be able to,

- Define software architecture.
- Distinguish between intentional and accidental architecture, recognizing that every software system has an architecture.
- Explain how software architecture is developed and evolved under plan-driven and Agile development approaches.
- Identify the factors that influence software architecture.

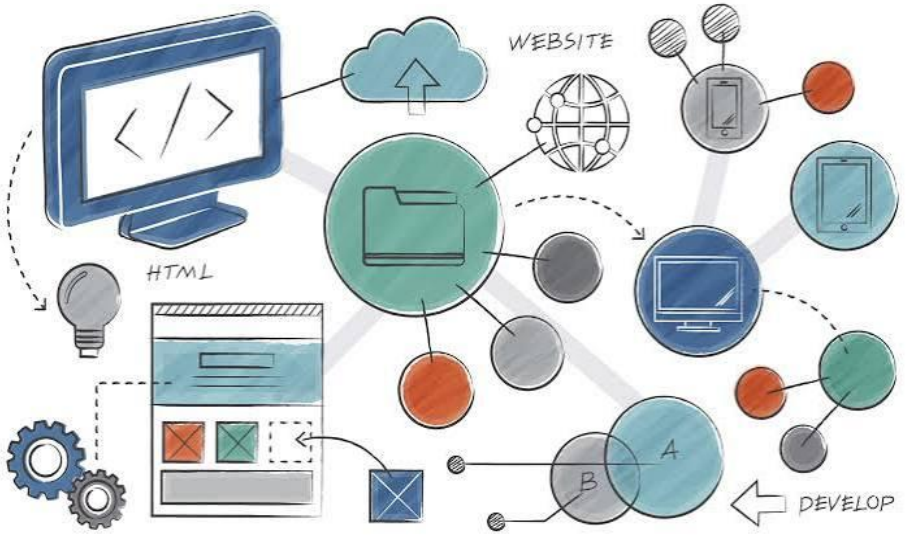
CONTENTS

WHAT IS SOFTWARE?

- Software enables a computer or computing system to perform a required task.
- It is developed to solve a problem or support a human or business activity.
- It receives inputs, processes information, maintains data and produces outputs.
- Software may operate independently or as part of a larger system.

SOFTWARE IS MORE THAN SOURCE CODE

- A software system may include,
  - Source code and executable programs
  - Data and databases
  - Configurations, Libraries and frameworks
  - External services
  - Interfaces to other systems
  - Deployment and runtime environments



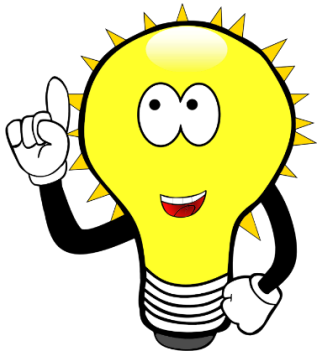
- These parts must work together as one system.

WHY DOES SOFTWARE BECOME COMPLEX?

- Software becomes complex because of,
  - Increasing functionality
  - More users and stakeholders
  - More modules, components and services
  - Relationships and dependencies among system elements
  - Integration with external systems
  - Quality expectations
  - Changing requirements and technologies
- The difficulty comes not only from the number of elements, but also from how they interact.

COMPLEXITY MUST BE MANAGED

- Complexity must be managed by,
  - Dividing the system into meaningful parts
  - Assigning clear responsibilities to each part
  - Establishing boundaries for the parts
  - Controlling dependencies among parts
  - Defining how the parts communicate
  - Applying consistent rules across the system



THE NEED FOR SOFTWARE ARCHITECTURE

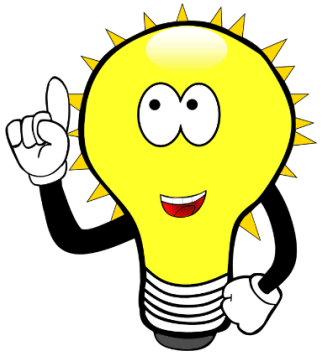
- Software architecture provides the high-level structure required to manage a complex system.
- It helps development teams determine,
  - What the major parts of the system are
  - What each part is responsible for
  - How the parts are connected and which dependencies are allowed
  - Which qualities the system must support
  - Which decisions must remain consistent
- **Architecture does not remove complexity**.
- It **organizes complexity so that the system can be understood, developed and changed**.

WHAT IS SOFTWARE ARCHITECTURE?

- Software architecture describes the fundamental structure and important decisions of a software system.
- Richards and Ford **define software architecture** through four dimensions.
  - **Quality Attributes (Architectural Characteristics)**
  - **Logical components**
  - **Architectural style**
  - **Architectural decisions**
- These dimensions are applicable throughout the system.

DEFINING SOFTWARE ARCHITECTURE: FOUR DIMENSIONS

- **Quality Attributes** (Architectural Characteristics)
  - The capabilities and qualities required for the system to succeed.
- **Logical components**
  - The major elements that implement the behaviour of the system.
- **Architectural style**
  - The overall structural arrangement used by the system.
- **Architectural decisions**
  - The rules and constraints that guide construction of the system.



FOUR DIMENSIONS OF SOFTWARE ARCHITECTURE

Architectural Dimension	Purpose	Examples
Quality Attributes (Architectural Characteristics)	Describe what the system must be capable of supporting and the conditions for success.	Performance, Scalability, Security, Availability
Logical Components	Divide system behaviour into meaningful parts and show where responsibilities belong.	Domains, Services, Workflows, Business capabilities
Architectural Style	Defines the general structural approach used to implement the system.	Layered, Modular monolith, Event-driven, Microservices
Architectural Decisions	Establish rules and constraints that guide system construction.	Data ownership, Communication mechanisms, Access rules, Security rules and regulations etc.

DEFINING SOFTWARE ARCHITECTURE: FOUR DIMENSIONS

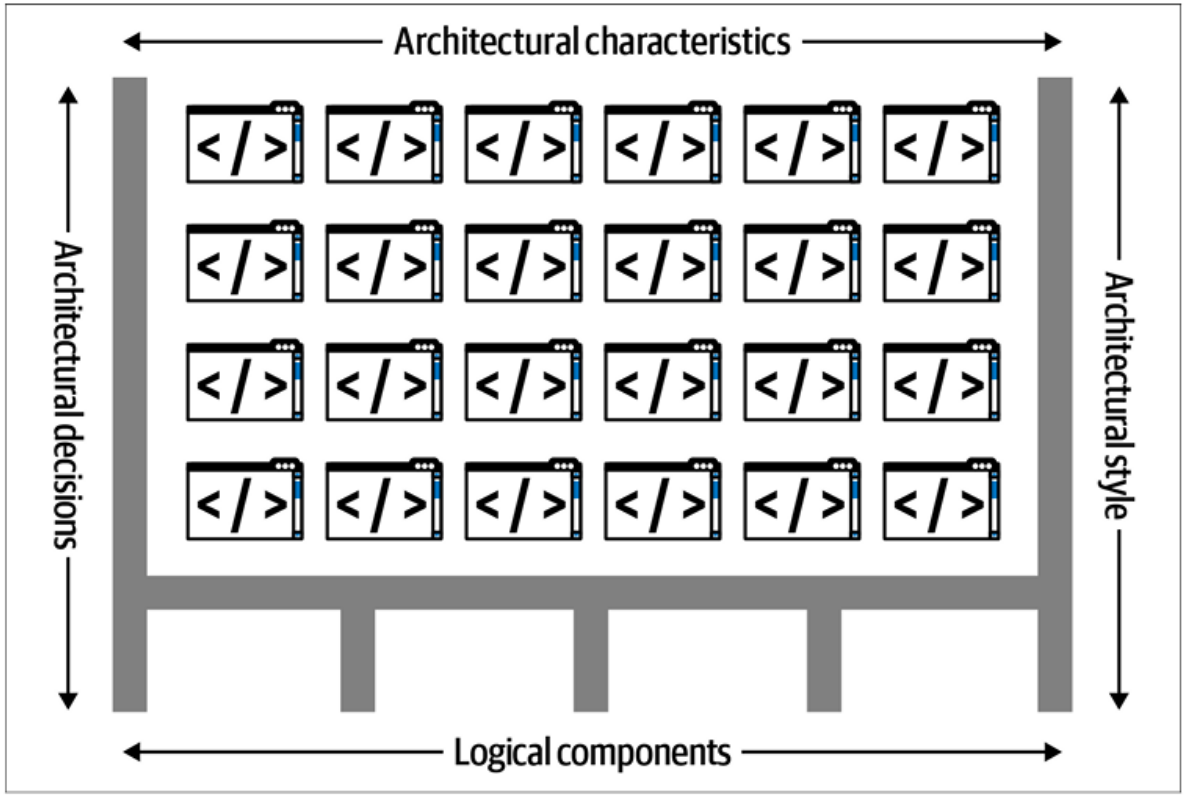
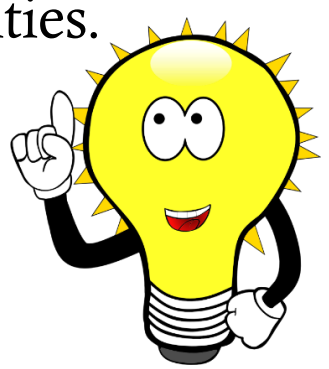


Figure 1-1. Architecture consists of the system's structure, combined with architecture characteristics ("-ilities"), logical components, architecture styles, and decisions

FOUR DIMENSIONS OF SOFTWARE ARCHITECTURE

- How do the dimensions connect to produce a software architecture?
  - Understand the problem domain.
  - Identify the important Quality Attributes (Architectural Characteristics).
  - Identify the logical components required to implement system behaviour.
  - Select an architectural style that supports the required structure and qualities.
  - Establish architectural decisions that guide implementation.
- A **style alone does not represent** the **complete architecture**.



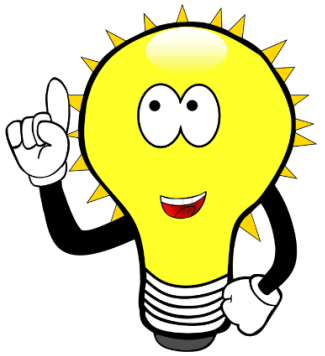
A RELATABLE EXAMPLE

ARCHITECTURE VS. DESIGN

- Architecture and design are not completely separate activities; they exist on a continuum.
- A decision becomes more architectural when it is,
  - Strategic rather than tactical
  - Difficult or expensive to change
  - Long-lasting
  - Relevant to several parts of the system
  - Associated with significant trade-offs
- Changing the Data Storage mechanism generally can be considered closer to an architectural decision.
- Detailed classes, algorithms and screen layouts are generally closer to design.

ARCHITECTURE DEPENDS ON CONTEXT

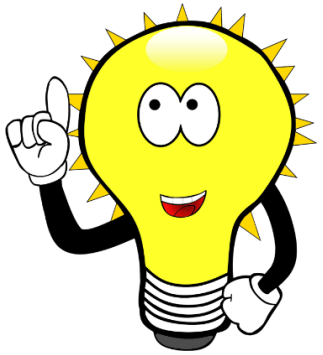
- Architectural decisions depend on many factors.
  - Business needs
  - Available technologies
  - Cost
  - Time
  - Development skills
  - Operational environment
  - Existing systems
  - Organizational constraints



- **An architecture that is suitable for one system may be unsuitable for another.**

ARCHITECTURE IS BASED ON TRADE-OFFS

- Everything in software architecture is a trade-off.
- For example,
  - Greater security may reduce usability.
  - Greater consistency may reduce availability.
  - Greater flexibility may increase complexity.
  - Greater isolation may increase communication overhead.
  - Faster delivery may increase technical debt.
- There is **no single best architecture** for every system.
- Usually, what we end up with is the **Least-worst architecture**.



DOES ALL SOFTWARE HAVE AN ARCHITECTURE?

- **Yes!**
- Any software system has,
  - Some form of structure
  - Elements with responsibilities
  - Relationships among those elements
  - Dependencies
  - Data flows
  - Rules, whether explicit or implicit

- The real question is whether the architecture has been **consciously understood and managed.**

INTENTIONAL ARCHITECTURE

- **Intentional architecture** results from **deliberate decisions**.
  - Important Quality Attributes (Architectural Characteristics) are identified.
  - Responsibilities are assigned clearly.
  - Dependencies are controlled.
  - An appropriate structural approach is selected.
  - Important decisions are communicated.
  - The architecture is reviewed as the system changes.
- Note: Intentional architecture **does not** mean that every decision must be made at the beginning.

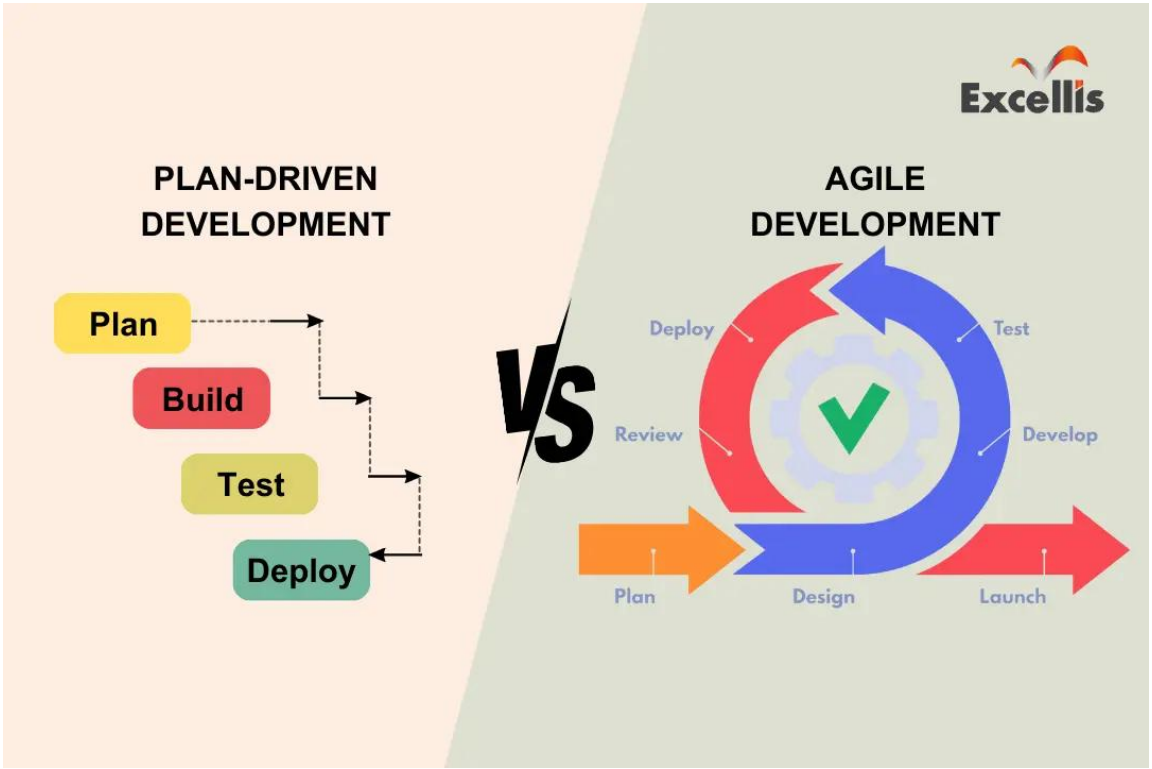
ACCIDENTAL ARCHITECTURE

- **Accidental architecture** emerges **without sufficient architectural reasoning**.
- It may result when teams,
  - Begin coding without considering the overall structure
  - Make isolated local decisions
  - Add dependencies whenever they appear convenient
  - Allow frameworks to determine the system structure
  - Copy existing solutions without considering the context
  - Delay important decisions until change becomes expensive
- **Not planning** the architecture **does not produce** a **system without architecture**.
- It produces **an architecture** that was **not deliberately selected**.

INTENTIONAL VS ACCIDENTAL ARCHITECTURE

Approach	How It Develops	Key Characteristics
Intentional Architecture	Structure is considered deliberately.	• Decisions are justified • Dependencies and boundaries are controlled • The architecture is reviewed as the system changes.
Accidental Architecture	Structure results from unrelated local decisions.	• Dependencies grow without control • Decisions may not be documented or understood • Problems become visible only when the system is difficult to change.

TWO APPROACHES TO INTENTIONAL ARCHITECTURE



Source: https://www.excellisit.com/differentiating-plan-driven-development-vs-agile-development/

TWO APPROACHES TO INTENTIONAL ARCHITECTURE

- **Plan-driven approach**
  - Most major architectural planning is performed up front.
  - E.g. Waterfall Methodology
- **Agile approach**
  - Initial architecture is established, and further planning is performed incrementally.
  - E.g. SCRUM
- Both approaches involve deliberate architectural decisions.

TWO APPROACHES TO INTENTIONAL ARCHITECTURE

- In a **Plan-driven approach,**
  - Plan most of the architecture before implementation.
  - Requirements are studied early.
  - The overall structure is planned before most implementation begins.
  - Major components and interfaces are identified early.
  - Development follows the planned architecture.
  - Significant changes may require the architecture to be revised.
- Architecture is treated as an early foundation for development.

TWO APPROACHES TO INTENTIONAL ARCHITECTURE

- In an **Agile approach**,
  - Establish enough architecture to begin and continue planning as the system develops.
  - Additional decisions are made as the system grows.
  - The architecture evolves through implementation and feedback.
  - Refactoring is used to improve the structure.
  - The development team participates in architectural decisions.
  - Important decisions are made when sufficient information becomes available.

TWO APPROACHES TO INTENTIONAL ARCHITECTURE

(Continued from **Agile approach)**

- Architecture is treated as a continuing activity.
    - An Agile architecture may emerge incrementally.
- However, this does not mean that it emerges without direction.
- Architecturally significant decisions must still be made deliberately.

WHAT DRIVES ARCHITECTURE?

- Architecture does not begin with selecting an architectural style.
- It begins by understanding important queries such as,
  - What the organization is trying to achieve?
  - What the system must do?
  - Which constraints cannot be ignored?
  - Which trade-offs are acceptable?
- The architecture must be justified by the needs of the system and its environment.

BUSINESS DRIVERS

- Business drivers are the business goals, pressures and constraints that determine the success of the system.
- Examples include:
  - Reducing time to market
  - Improving user satisfaction
  - Gaining a competitive advantage
  - Meeting a fixed deadline
  - Remaining within a limited budget
- Business drivers are normally expressed in business language.

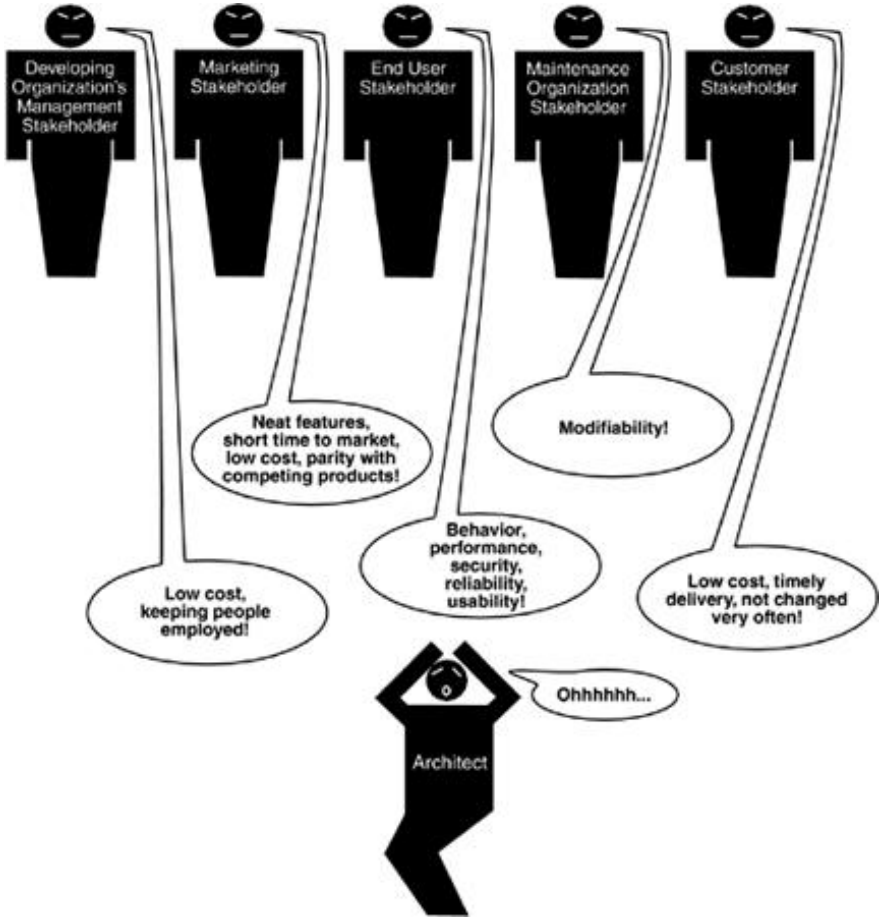
BUSINESS DRIVERS

- Architects must translate business concerns into Quality Attributes (Architectural Characteristics).
- For example, “the company expects rapid growth” could mean Scalability, Elasticity, Availability, Deployability should be prioritized quality attributes.
- The architect must understand what the business wants and identify the technical qualities needed to support it.
- It is not practical to treat every Quality Attribute as equally important.
- Prioritizing Quality Attributes and identifying trade-offs are necessary.

THE ARCHITECTURE BUSINESS CYCLE

- Business drivers explain what the organization wants the system to achieve.
- However, architecture is also influenced by many factors including,
  - Stakeholders
  - The developing organization
  - Existing systems and assets
  - The architect's experience
  - Available technologies and resources
  - Industry practices
  - Schedule and budget
- The **Architecture Business Cycle** provides a broader view of these influences.

THE ARCHITECTURE BUSINESS CYCLE

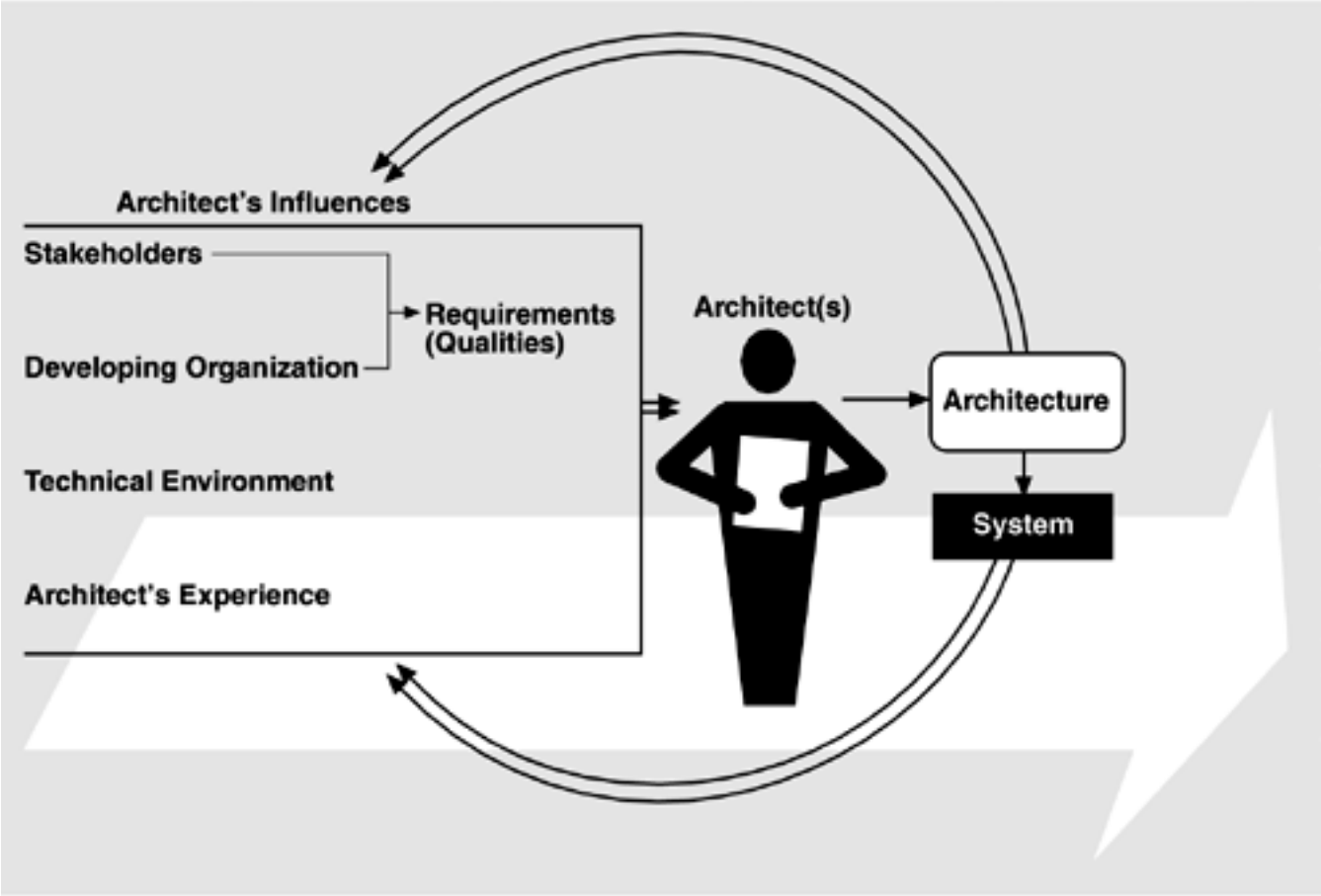


Source: https://people.ece.ubc.ca/~matei/EECE417/BASS/ch01lev1sec1.html

THE ARCHITECTURE BUSINESS CYCLE

- The Architecture Business Cycle describes the relationship between architecture and its environment and how architecture itself influences that environment in return.
- Software architecture is influenced by factors such as,
  - Stakeholder concerns
  - Developing Organization
  - Technical Environment
  - Architect's Experience
- The resulting architecture and system then influence those environments in return.
- Further Reading: Refer this link.

THE ARCHITECTURE BUSINESS CYCLE



Source: https://people.ece.ubc.ca/~matei/EECE417/BASS/ch01lev1sec1.html

SUMMARY

- Software becomes complex as functionality, elements, dependencies and quality expectations increase.
- Complexity must be managed through structure, boundaries, responsibilities and decisions.
- Software architecture consists of Quality Attributes (Architectural Characteristics), logical components, architectural style and architectural decisions.
- Every software system has an architecture.
- Architecture may be intentional or accidental.
- Intentional architecture may be developed largely up front or incrementally.
- Agile development does not remove the need for architecture.
- Business drivers are translated into Quality Attributes (Architectural Characteristics).
- The Architecture Business Cycle explains the wider influences to and from architecture.

REFERENCES

- M. Richards and N. Ford, “Fundamentals of Software Architecture: A Modern Engineering Approach,” 2nd ed. Sebastopol, CA: O’Reilly Media, 2025.
- L. Bass, P. C. Clements, and R. Kazman, “Software Architecture in Practice,” 3rd ed. Boston, MA: Addison-Wesley Professional, 2012.